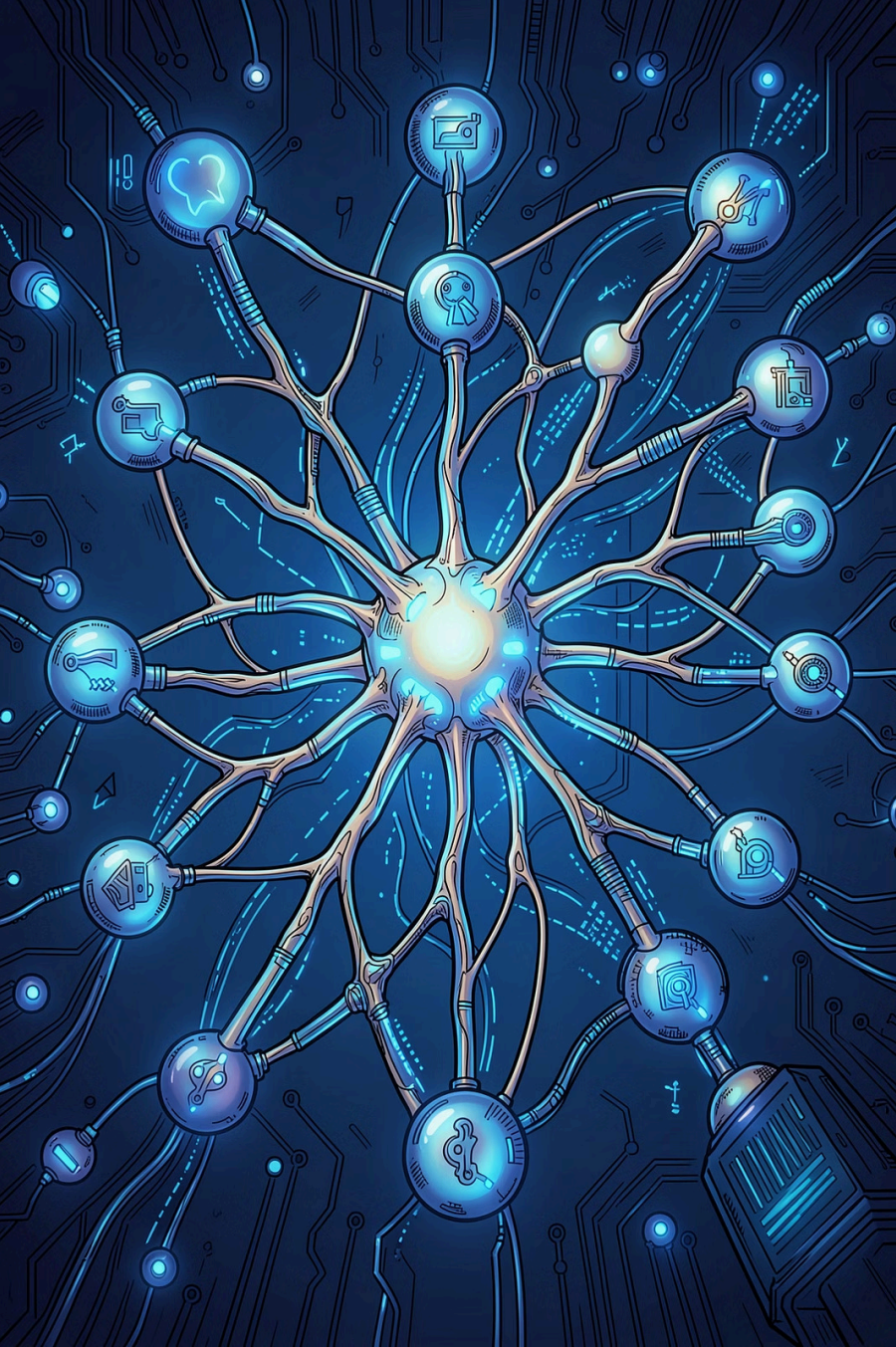




# ארכיטקטורת סוכנים ותשתית MCP

סיכום יחידת לימוד — מושגים, רכיבים, ועקרונות עיצוב לבניית מערכות AI חכמות, מאובטחות ויעילות

# המושגים המרכזיים: מי הוא מי?

## פרוטוקול MCP

ה"צינור" האוניברסלי המאפשר לסוכני AI להתחבר בצורה מאובטחת למקורות מידע מקומיים, בסיסי נתונים וכלים מורכבים.

## תת-סוכן (Sub-Agent)

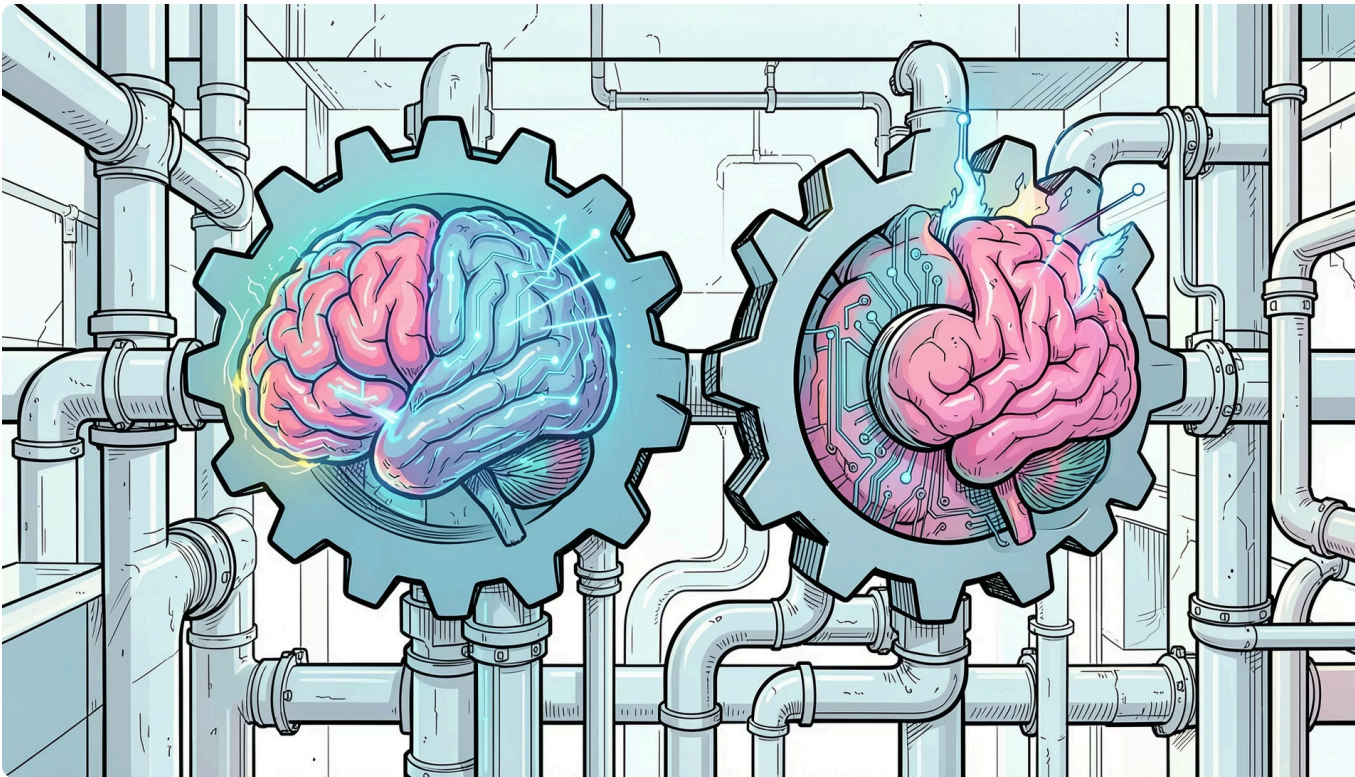
סוכן קטן וממוקד למשימה ספציפית. הפרדת משימות לתתי-סוכנים מונעת פיזור, מעלה איכות תוצרים וחוסכת משאבים.

## סוכן (Agent) AI

ישות AI אוטונומית עם תפקיד, יעדים וחוקים. מסוגלת לקבל החלטות בזמן אמת, להפעיל כלים חיצוניים ולנהל תהליך עבודה שלם.

# שתי הספריות: מוח ותשתית

שילוב שתי ספריות בעלות תפקידים משלימים — אחת מספקת את החוכמה, השנייה את הצינור.



## ספריית המוח — openai

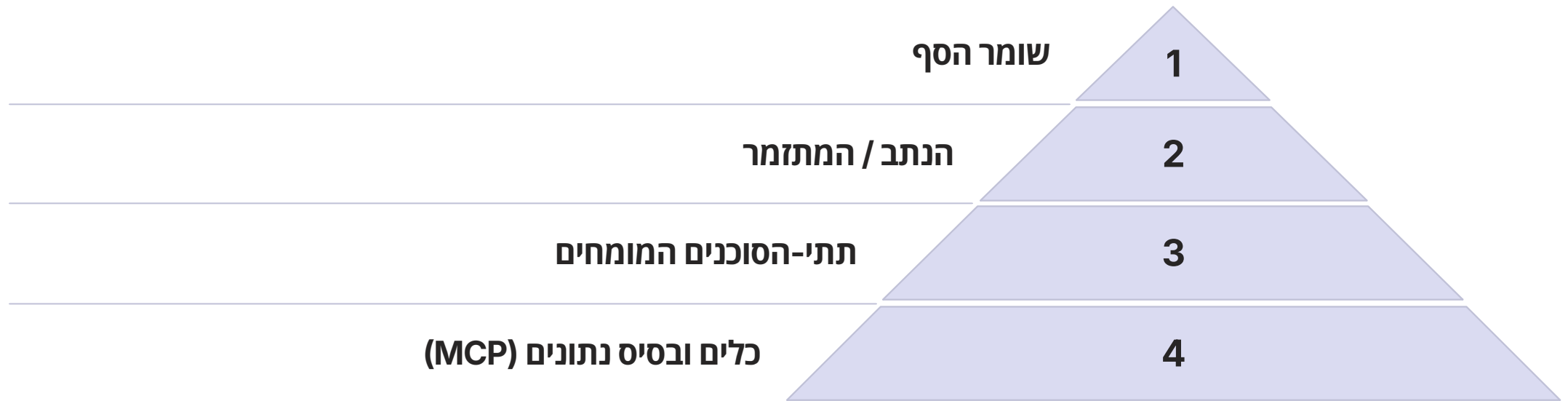
לשיחה עם מודלי שפה כמו SDK  
אחראית על ChatGPT או DeepSeek  
החלקים החכמים — ניתוח, החלטות  
ואינטליגנציה.

## @modelcontextprotocol/sdk — ספריית הצינור

הופכת את הקוד לשרת תקשורת מקומי.  
מצהירה לקליין אילו כלים קיימים ומאפשרת  
העברת פרמטרים ותשובות.

# ארכיטקטורת הסוכן: 4 שכבות קבועות

מערכת סוכנים Production-Ready בנויה מארבע שכבות מובנות — כל אחת ממלאת תפקיד ייחודי בשרשרת העיבוד.



השכבות עובדות יחד כשרשרת: כל בקשה עוברת סינון, ניתוב, עיבוד עמוק, וגישה למידע — בסדר קבוע ומוגדר.

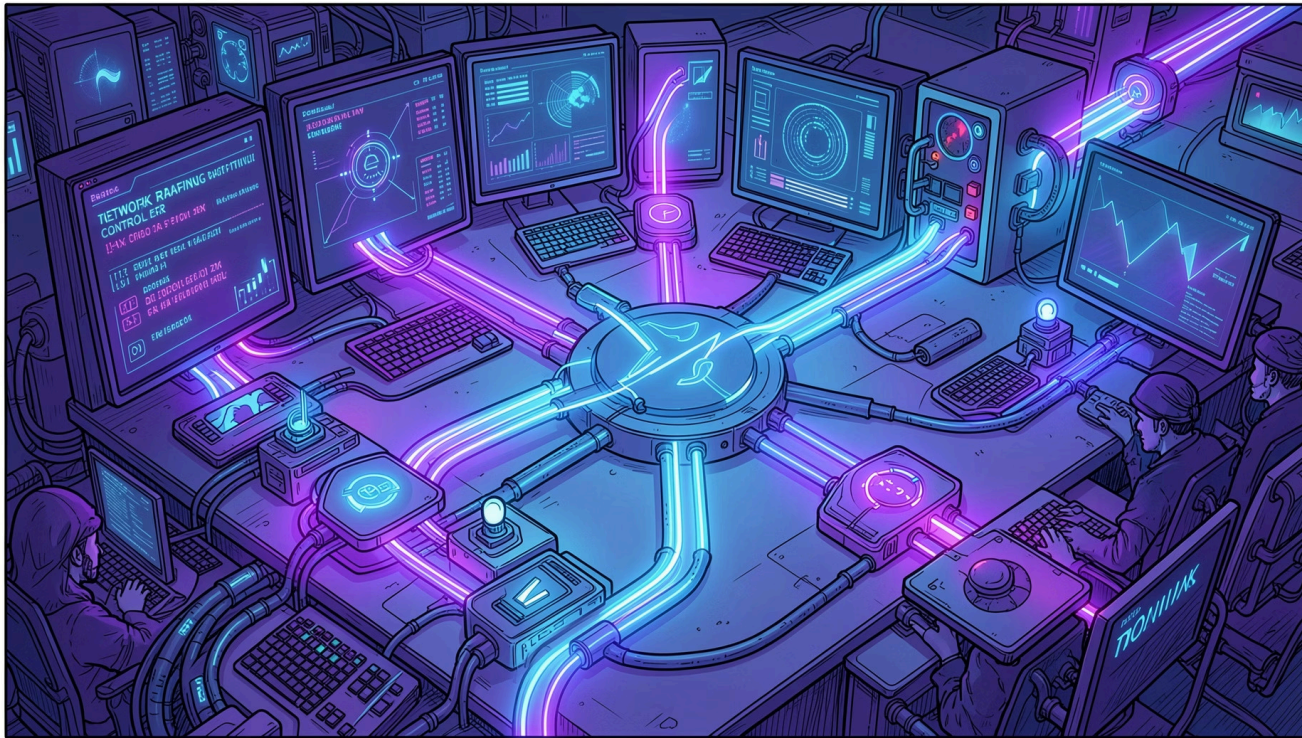




# שכבה 1: שומר הסף (Guardrails)

השכבה הראשונה בכניסה — מסננת בקשות מסוכנות, נושאים מחוץ לתחום, וניסיונות **Prompt Injection**. בקוד, שומר הסף הוא **חומה קשיחה** שה-AI לא יכול לעקוף.

# שכבה 2: הנתב / המתזמר (Router)



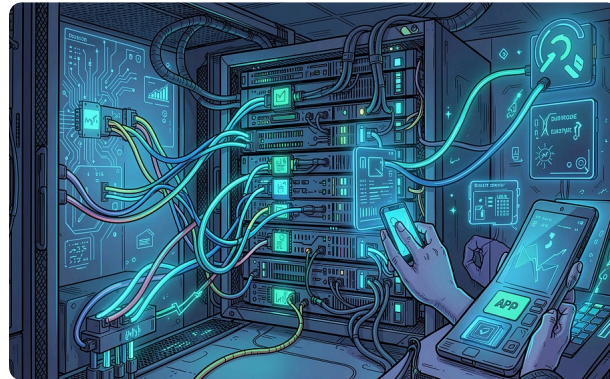
## המוח הניהולי של המערכת

הנתב מנתח כל בקשה נכנסת ומחליט באופן דינמי לאיזה נתיב או תת-סוכן להעביר את העבודה.

מזהה את כוונת המשתמש וסוג השאלה → מפנה לתת-סוכן המתאים (למשל: מומחה מגזר חרדי vs. כללי) → מנהל את זרימת המידע בין השכבות

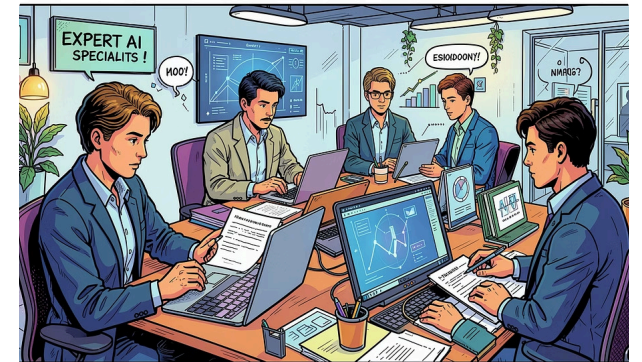


## שכבות 3-4: מומחים, כלים ובסיס נתונים



### כלים ובסיס הנתונים (MCP)

המידע הקשיח והקוד — כמו מאגר המשפחתונים — שהסוכנים נעזרים בו כדי לקבל החלטות מבוססות עובדות בזמן אמת.



### תתי-הסוכנים המומחים

מוחות ה-AI שמבצעים את עבודת העומק — כל תת-סוכן מתמחה בתחום ספציפי ומספק תשובות מדויקות וממוקדות.

# חתי No-Code וחתי Code-Based?

פלטפורמות כמו Copilot ו-Custom GPTs מאפשרות יצירת סוכנים בפרומפטים בלבד — אבל יש מצבים שמחייבים קוד.



## 🔧 No-Code — כשמתאים

- מהירות פיתוח — עוזר דיגיטלי תוך 5 דקות
- מידע סטטי וקבוע כמו תקנון בית ספר
- תהליכים פשוטים ללא לוגיקה מורכבת

## 💻 Code-Based — כשחייבים

- יציבות מוחלטת בזרימה — חוקים נוקשים שלא ניתן להפר
- חיבור לבסיסי נתונים פנימיים ומאובטחים
- אבטחה מלאה — ה-AI לא יכול לעקוף את החוקים



# הבונוס הגדול: חיסכון אדיר בטוקנים

## ❌ No-Code — כדור שלג

בכל הודעה נשלחים מחדש **כל** הפרומפטים של **כל** תתי-הסוכנים + היסטוריית השיחה.  
העלות קופצת בריבוע.

## ✅ Code-Based — דיאטה חכמה

שומר הסף מקבל רק פרומפט קטן. אם בסיס הנתונים החזיר JSON ענק — מחשבים  
ממוצע ב-JavaScript ושולחים **רק תוצאה אחת**. פחות טוקנים = פחות כסף!



# סיכום: מתכנתת AI אמיתית

הפרומפט הוא ההנחיות והאופי שנותנים לעובד. **הקוד** הוא פס הייצור, מערכת המחשוב והארנק של החברה.

מתכנתת AI אמיתית יודעת לשלב את שניהם — כדי לבנות מוצר **חכם, מאובטח וזול**.

01

## הגדרת ארכיטקטורה

4 שכבות: Guardrails → Router → Sub-Agents → Tools/MCP

02

## בחירת כלי פיתוח

+ לאינטליגנציה openai  
לתשתית @modelcontextprotocol/sdk

03

## החלטת אסטרטגיה

ליציבות, Code-Based, למהירות No-Code  
אבטחה וחסכון